



SMART FIT

Votre garde-robe virtuelle intelligente

Sommaire

01

Présentation du projet
et objectifs

02

Choix techniques

03

Réalisation

04

Difficultés

05

Perspectives



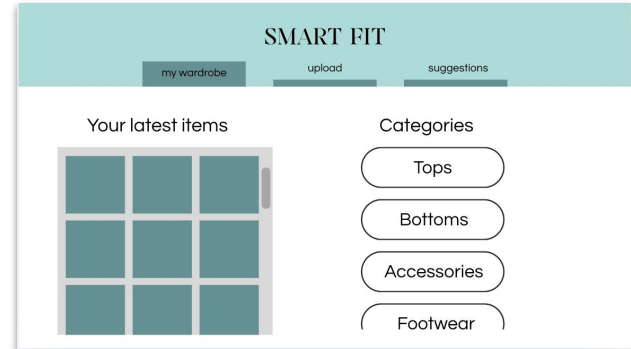
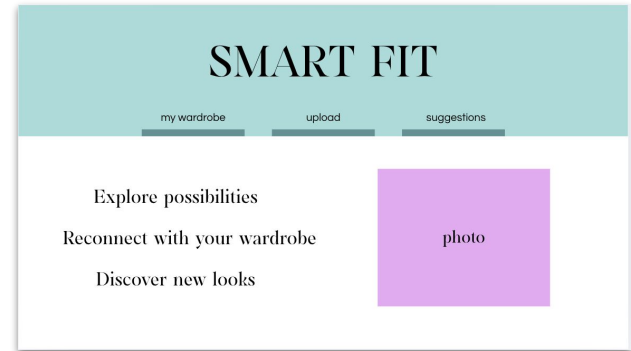
01

Présentation du projet et objectifs

Présentation du projet

Garde robe virtuelle

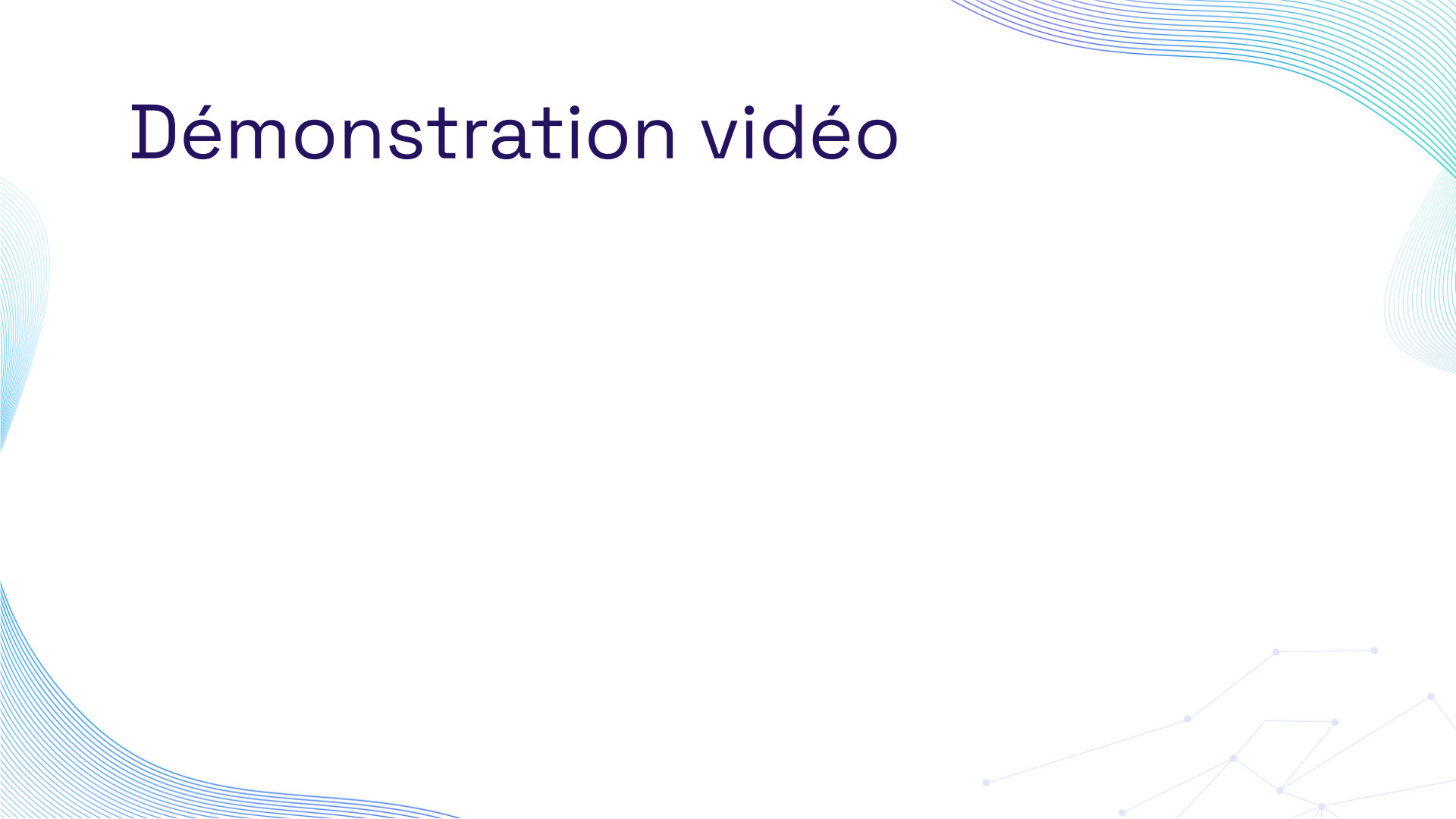
- L'utilisateur peut y ajouter des photos de ses vêtements
- Les vêtements sont classés par catégorie / couleur
- SmartFit peut proposer des suggestions en fonction de la météo ou des envies de l'utilisateur



Objectifs initiaux

- Développer un modèle de classification d'images
- Intégrer ce modèle au sein de l'interface WEB
- Implémenter des recommandations intelligentes
- Permettre à l'utilisateur de créer un compte, et donc de retrouver les photos importées précédemment

Démonstration vidéo



SMART FIT

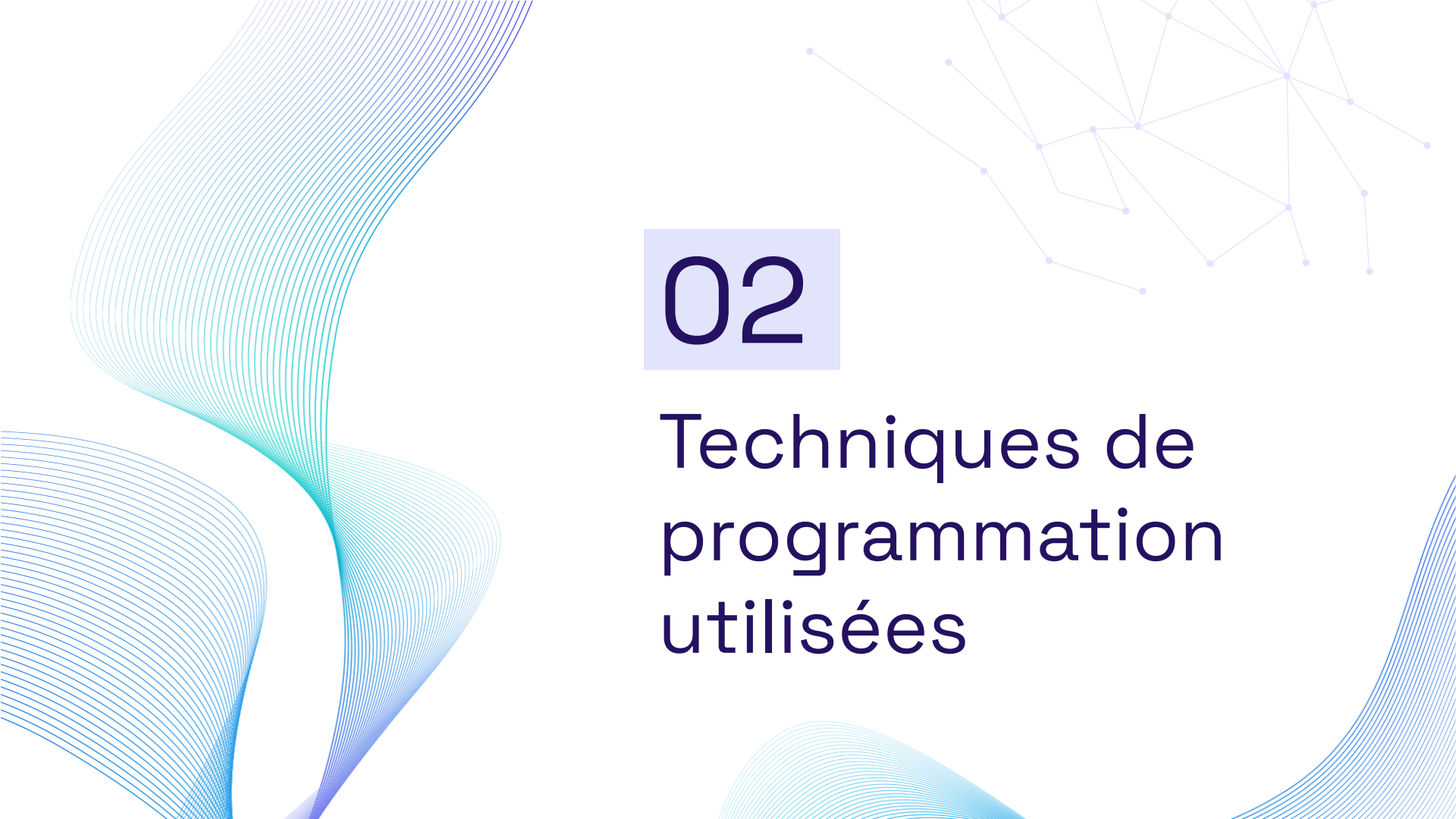
[Home](#)[Login](#)[Sign Up](#)

Explore possibilities

Reconnect with your wardrobe

Discover new looks





02

Techniques de programmation utilisées

Techniques de programmation utilisées

Front End HTML/ CSS

- Pas besoin d'installer des outils complexes supplémentaires
- Temps de chargement plus faible car le navigateur n'a pas à importer de lourdes bibliothèques
- Cohérence avec l'architecture Flask, qui est conçu pour fonctionner avec Jinja2
- Contrôle total du design

Techniques de programmation utilisées

Implémentation Python

- Flask :
 - Framework léger, intègre facilement les scripts Python complexes sans contraintes structurelles importantes
- SQLAlchemy :
 - manipulation d'objets Python plutôt que d'écrire en SQL : plus lisible et moins sujet aux erreurs , bonne portabilité



03

Réalisation du site internet

Système d'analyse du vêtement

1. Detection de la categorie
2. Détection de la couleur
3. Confirmation de l'utilisateur
4. Stockage dans la base de donnée
5. Suggestions

Backend

- Model training
- predict_eng_model.py (prédiction du modèle + couleur)
- recommander.py (obtenir les suggestions)
- database.py : Classe Clothing (base de donnée)

Static

- uploads (images)
- app.py (gestion Flask)

a. Reconnaissance des vêtements

Base de données :

Crowdsourced database

5404 images 20 classes

5097 images utilisables

4724 parmi les classes choisies, avec au moins 100 photos
(jusque 1011 pour t-shirt)



Travail en français, puis en anglais :

Chapeau, Chaussures, Chemise, Jupe, Manteau, Pantalon, Pull, Robe, Short, T-shirt, Veste

blazer, dress, hat, hoodie, long sleeve, outerwear, pants, shirt, shoes, shorts, skirt, t-shirt

Data splitting : train (80% 3779 images), val et test sets

a. Reconnaissance des vêtements

2 méthodes :

- Entraînement de zéro : rapidement abandonné
- Fine tuning model Resnet 18 : transfer learning

1. Chargement du modèle :

ResNet18 pré-entraîné sur ImageNet (1000 classes diverses).

Structure ResNet18

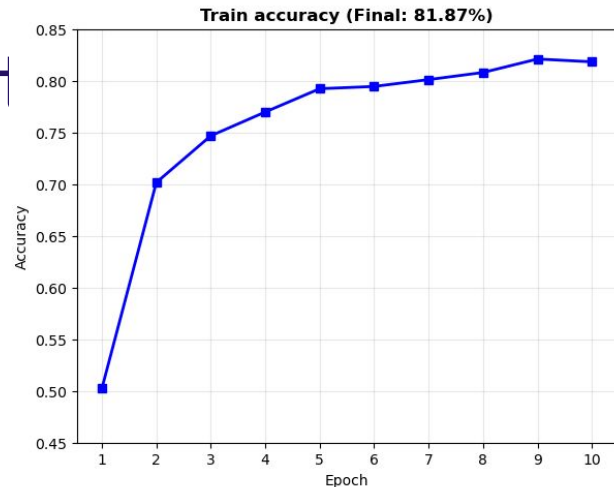
- Convolution initiale + Max Pooling
- 4 blocs (layer1, layer2, layer3, layer4) qui augmentent en complexité de reconnaissance
- Average Pooling global
- Couche fully connected

a. Reconnaissance des vêtements

2. **Nouvelle** dernière couche linéaire : prédiction de 12 classes.

Val_accuracy = 76.89%

Palier époque 5

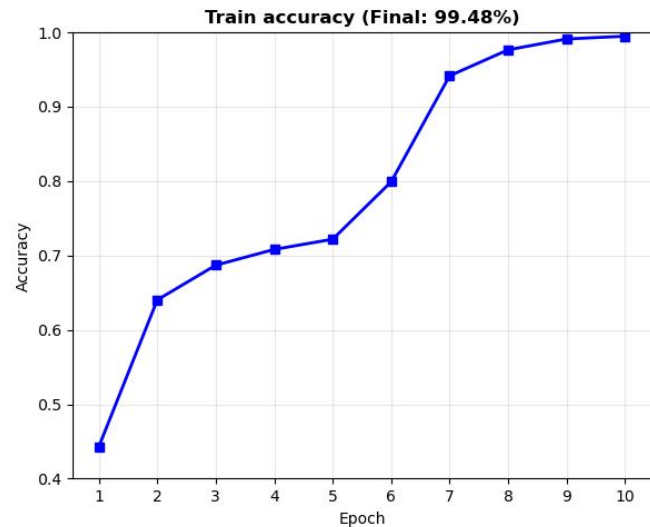


3. A la 5ème époque, **libération de la couche 4**

```
for param in model.layer4.parameters():  
    param.requires_grad = True
```

-> overfitting

mais Val_accuracy = 86.04% (+10% !)



a. Reconnaissance des vêtements

Imprévus :

- Version française puis anglaise
- Taille des fichiers :
 - Fichier original : 2Mo / image →
Training time : 19min29s
 - Fichiers compressés : 20ko/image →
Training time : 2min28s→ gain en rapidité sans perte d'efficacité
- Modèle encore améliorable : temps passé sur le reste
- T-Shirts = $\frac{1}{3}$ de la base de données



Color detected: **Black**

The model detected : **pants**. Is it correct ?

Final category :

Pants (Detected)



b. Détection de la couleur

K-means :

- Rogner le contour
- Redimensionnement 60x60 pixels : léger
- K-means global :
 - Clustering 3 couleurs RGB principales de l'image

Exemple typique

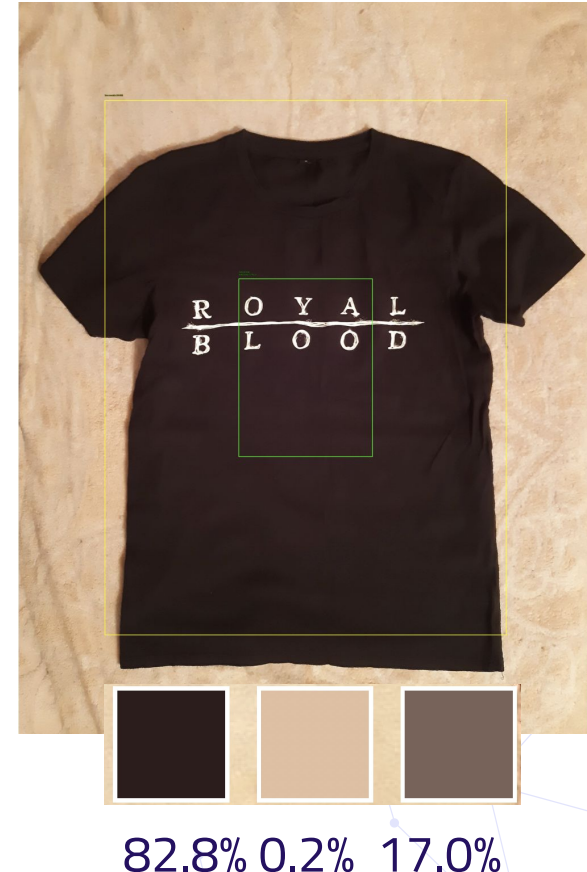


b. Détection de la couleur

- K-means central :
 - Si +2 couleurs pcg au centres → multicolor
- Correspondance des couleurs :
 - Passage en LAB approximé (+ léger)
 - Prend la couleur de référence la plus proche

```
COLOR_MAP = {  
    "Blue": (0, 70, 170),  
    "Red": (200, 0, 0),  
    "Orange": (255, 140, 0),  
    "Yellow": (255, 230, 0),  
    "Green": (0, 100, 0),  
    "Purple": (100, 0, 100),  
    "Pink": (255, 150, 180),  
    "White": (245, 245, 245),  
    "Black": (20, 20, 20),  
    "Grey": (120, 120, 120),  
    "Beige": (225, 200, 170),  
    "Navy": (0, 0, 80)  
}
```

Exemple typique



b. Détection de la couleur

Difficultés:

- Déterminer la stratégie
- Encore des incohérences

Mais :

- Base de donnée de départ robuste



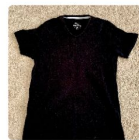
→ multicolor

C. Recommendations

Uniquement par couleur

→ Requête SQL

Your Outfit (Black)



Validation utilisateur

- La fonction upload appelle detect_clothing et detect_color
- Flask -> afficher upload.html avec formulaire de confirmation
- Une fois que l'utilisateur valide, le vêtement est enregistré avec sa couleur et son type

```
#systematic confirmation
return render_template(
    'upload.html',
    predicted_category=category,
    filepath=filepath,
    predicted_color=item_color,
    show_confirmation=True, #display the confirmation form
    categories=CATEGORIES,
    colors = COLORS
)
```

Blue
Red
Orange
Yellow
Green
Purple
Pink
White
Black
✓ Grey (Detected)
Beige
Navy
Multicolor

Step 2 : Confirm category



Color detected: **Grey**

The model detected : **outwear**. Is it correct ?

Final category :

Outwear (Detected)

Correct color :

Grey (Detected)

Confirm and add to the wardrobe

Création de compte pour l'utilisateur

- **Flask - login** : gestionnaire de session, gère le cycle de vie de l'authentification
- **Bibliothèque Werkzeug.security** : bibliothèque cryptographique, fournit les fonctions de hachage

Création de compte pour l'utilisateur

Fichier *database.py*

Classe *User*

- ID (clé primaire)
- Username
- Password
- Clothes

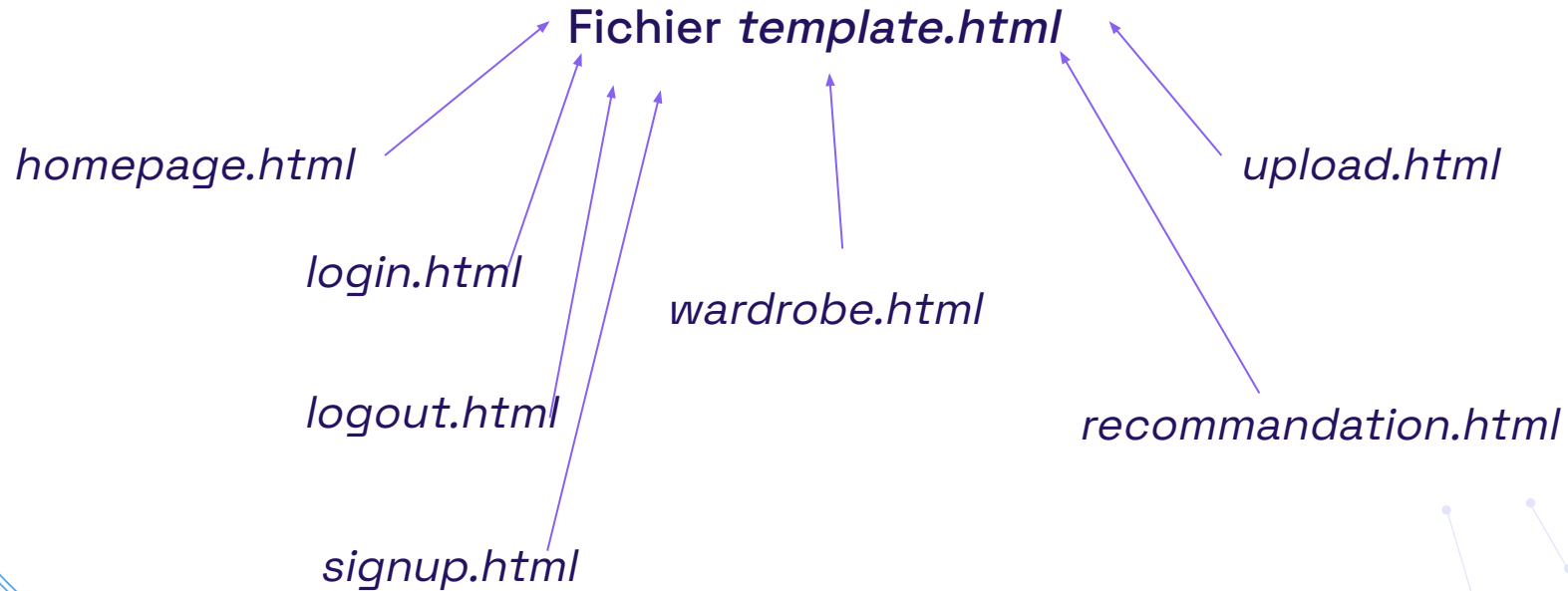
Classe *Clothing*

- User ID

Fichier *app.py*

- Werkzeug hache le mot de passe, SQLAlchemy l'enregistre
- Flask - login crée une session si le mot de passe est validé
- Décorateur *@login_required* sur les fonctions *upload* et *wardrobe*

Architecture des fichiers HTML



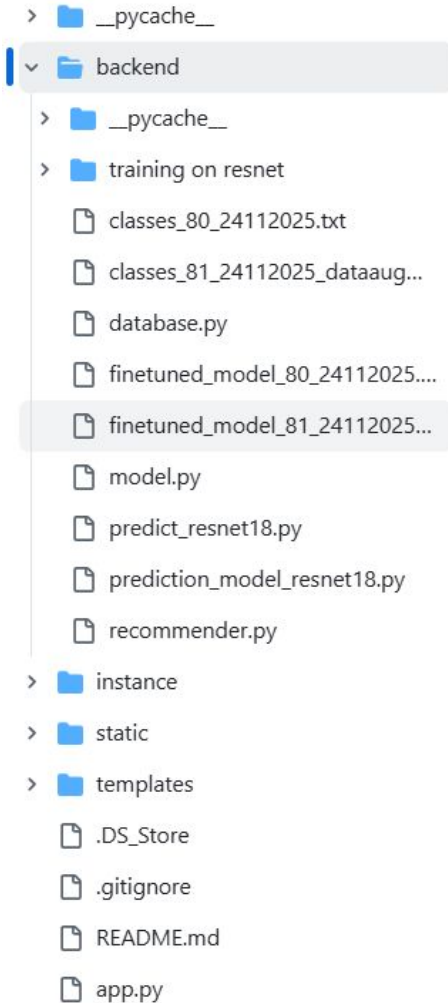


04

Difficultés rencontrées

a. Coder ensemble

- Répartir le travail
- Propreté et homogénéité du code (commentaires, architecture, noms de fichiers, savoir où trouver les données)
- Conflits de "merge"
- Suppression de la base de donnée à chaque merge



b. Prendre en main de nouveaux outils

- Nouveau langage: html
- Apprendre à utiliser flask

L'IA a été utile pour débloquer des bugs et comprendre le fonctionnement de nouveaux langages ou modules.

Néanmoins, il est important de toujours comprendre à 100% le code qu'on upload sur github

c. Traduction de nos objectifs initiaux en code

Des objectifs ambitieux pour un premier projet

- L'utilisateur se connecte pour accéder à sa garde robe
→ nécessite de coder à la fois le système de compte et de modifier le stockage des images
- Tri par couleur, par météo, historique des tenues
→ nécessitent des implémentations supplémentaires, pas toujours pertinent

d. S'adapter

Des besoins apparaissent au cours du projet

- Adapter les objectifs
- Option suppression des vêtements
- Confirmation de la prédiction de l'IA (pouvoir modifier si elle se trompe)



05

Perspectives d'amélioration et de développement

Possibilités de développements:

- Suggestions plus intelligentes
- Gestion de la confiance du modèle de reconnaissance :
 - Tri décroissant des catégories et couleurs
- Gestion Cloud des données plutôt qu'en local
- Connexion à un API météo,

Retirer le fond des photos,

Clothing segmentation : A partir d'une photo d'une tenue, séparer les différents vêtements



Merci !



Annexes

Premier modèle

Utilisation de PyTorch : plusieurs pistes

- Réseau linéaire "classique" : 3 couches linéaires + non linéarité entre chaque couche
 - CNN : 2 couches convolutionnelles avec pooling entre les deux + 2 couches linéaires + non linéarité
- Trop lent

Premier modèle

Performances du modèle 1 :

Entraîné sur 5 époques :

Accuracy: 64.7%, Avg loss: 1.091262

Pred: Ankle boot
Actual: Ankle boot



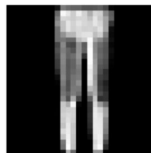
Pred: Pullover
Actual: Pullover



Pred: Trouser
Actual: Trouser



Pred: Trouser
Actual: Trouser



Pred: Pullover
Actual: Shirt



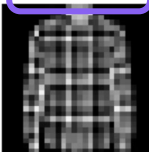
Pred: Trouser
Actual: Trouser



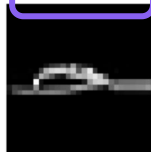
Pred: Coat
Actual: Coat



Pred: Pullover
Actual: Shirt



Pred: Sneaker
Actual: Sandal



Pred: Sneaker
Actual: Sneaker

